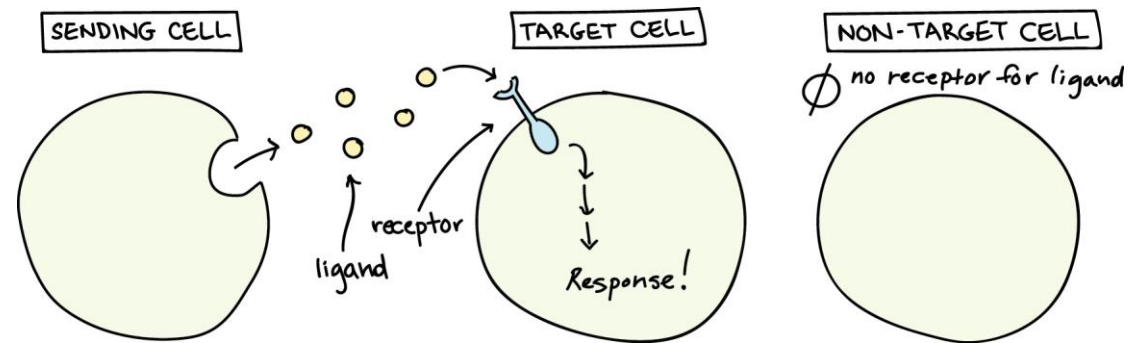


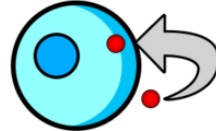
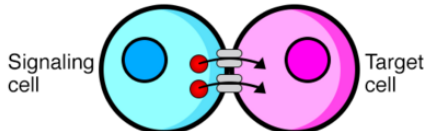
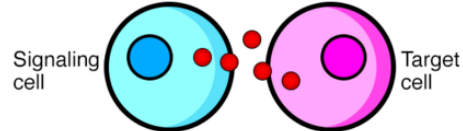
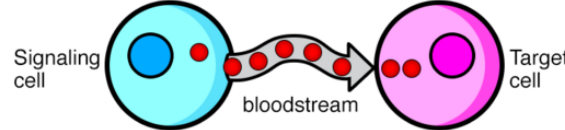
CHAPTER 2

Object-oriented
programming

Object-oriented programming



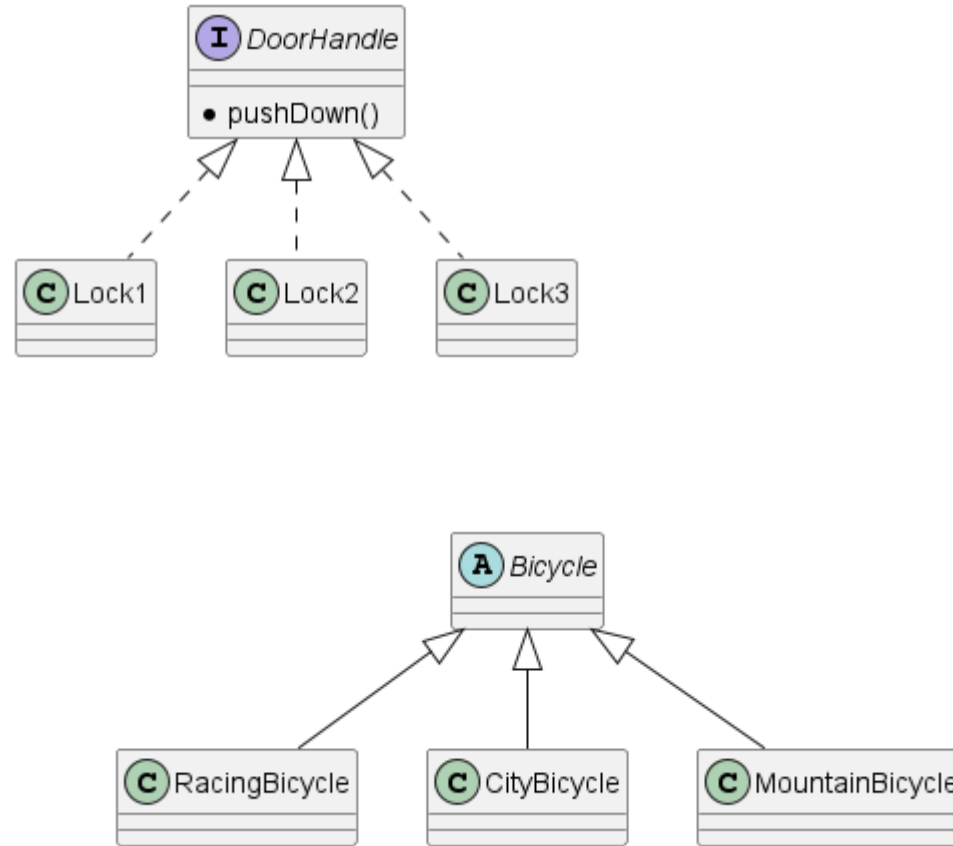
Object-oriented programming

Forms of Chemical Signaling	
Autocrine	A cell targets itself.
	
Intracrine	A cell targets receptors inside the cytoplasm or nucleus
	
Justacrine	A cell targets a cell connected by gap junctions.
	
Paracrine	A cell targets a nearby cell.
	
Endocrine	A cell targets a distant cell through the bloodstream.
	

Object-oriented programming

- The object-oriented paradigm is based on the concept of **objects**, which contain data and the code that manipulates on data together.
- During the object-oriented design process the concepts and elements of the modeled domain are abstracted as **objects**, which communicate by sending **messages** to each other.
- The messages that an objects can consume are described in the **contract** of the object.
- The objects "**encapsulates**" their attributes and the methods that manipulate on those attributes.
- Further object may be created based through the technique of "**inheritance**".
- The inheritance also serves as one form of "**polymorphism**", which makes different objects replaceable and developable independently.

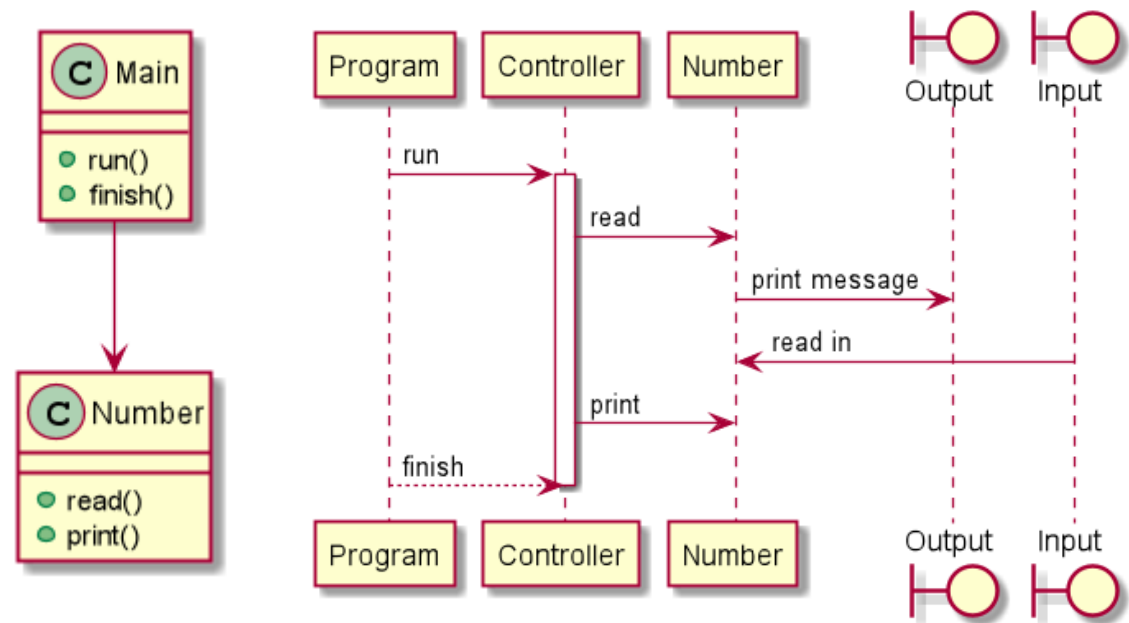
Object-oriented programming



Object-oriented programming

- In imperative object-oriented programming languages, the data is in the form of **fields**, and the code is stored in the form of functions and procedures.
- In object-oriented terminology the fields are called **attributes**, and the procedures are called **methods**.
- In these languages the messages are implemented as **method calls**, and the response messages appear as the **return values** of those methods.

Object-oriented programming



History

- 1958. The terminology of objects and attributes first appeared in relation with LISP at MIT.
- 1962-1967. Simula I and 67. (by Ole-Johan Dahl and Kristen Nygaard). It introduced **objects**, **classes**, **inheritance** and virtual procedures.
- 1971. Smalltalk (by Alan Kay etc.). The first pure OO language. It introduced the concept of **encapsulation**.



- 1985. C++ (by Bjarne Stroustrup). The object oriented extension of C++. It introduced the most widespread model of **class-based inheritance**, **object instantiation** and **encapsulation**. Its **exception handling** method was also influenced many languages.
- 1986. Eiffel (by Bertrand Meyers), declarative OO language that supports **design by contract**.
- 1986. Object Pascal (by Anders Hejlsberg).
- 1995. Delphi (by Anders Hejlsberg, Borland). Rapid Application Development (RAD) framework. The concept of **properties** was introduced in that.
- 1995. Java (by James Gosling). Besides having synthesized the earlier experience it introduced **interface inheritance**.
- 1995. JavaScript (by Brendan Eich). Multi-paradigm language, that supports **prototype-based inheritance**.



Encapsulation

- The ideas of encapsulation and message passing were raised first by **Alan Kay** who was inspired by how **cells** were encapsulated by a membrane and communicated by messages.
- The main purpose of encapsulating the data and the behavior inside objects was solving the problem of **race conditions**.
- Another important advantage of encapsulation is that the programmers could ignore the **implementation details**.
 - The objects are **replaceable** as long as they are hidden behind the proper interface.
 - Encapsulation is a precondition of **polymorphism**.

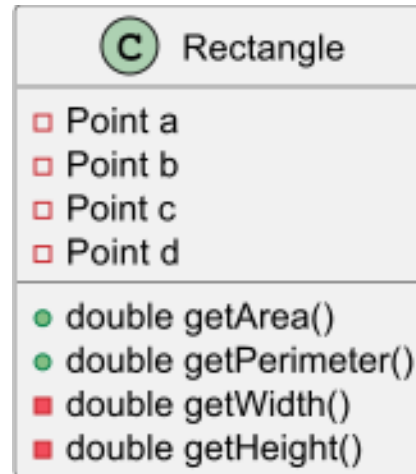
Encapsulation

- A **contract** is an agreement in which the sender and the receiver objects accept obligations for the content, the sending time and the effect of the message.
 - The **preconditions** contains restrictions for the states that the object must hold when the message is sent and its content.
 - The **postconditions** contains the states the object must hold after the message and the expectable response messages.
- An **invariant** is a property that holds at all times, except during the process of a message.

Encapsulation

- The most widespread model of encapsulation is the usage of **access declarations** as was introduced in C++. This solution is implemented in Object Pascal, **Java** and C#.
- There are languages (Smalltalk, Ruby) that enable access to the attributes **only through methods**.
- **Prototype-based languages** (like JavaScript) provide alternative methods for encapsulate data.

Encapsulation

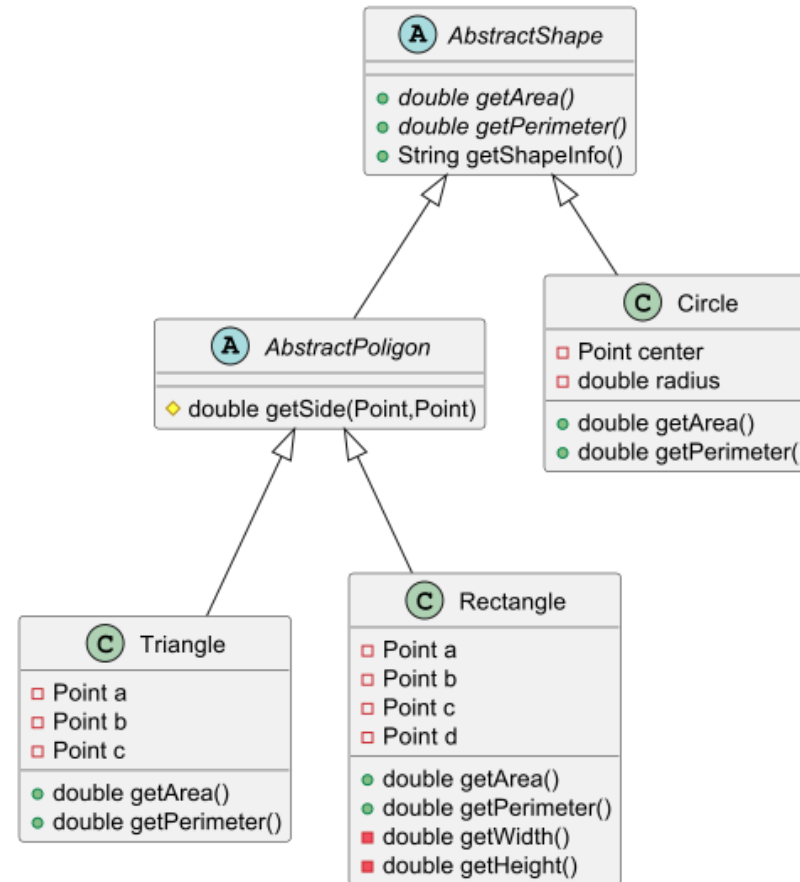


Inheritance

- Inheritance is the technic of creating objects based on existing objects.
- The main goals of inheritance are
 - **Re-using** the already implemented **source code** (implementation inheritance),
 - Provide **polymorphic** access to the object (interface inheritance).
- In general, there are **two types** of inheritance (consequently of languages)
 - **Class-based** inheritance (static).
 - **Prototype-based** inheritance (dynamic).

Inheritance

Class-based inheritance



Inheritance

Class-based inheritance

- It is based on **generalization**: the designer **classify object** based their common characteristics.
- The programmer creates the classes first. The objects will be the **instances** of the classes.
- New classes can be constructed **from the existing classes** through inheritance.
- We build **class hierarchies** by implementation and interface inheritance.
- Classes can be **final** (that can be not inherited further) and **abstract** (that can't be instantiated).
- This inheritance is **static**: the classes are types that can be checked by the compiler.
- Most OO languages (including **Java**) implement this kind of inheritance.

Inheritance

Class-based inheritance

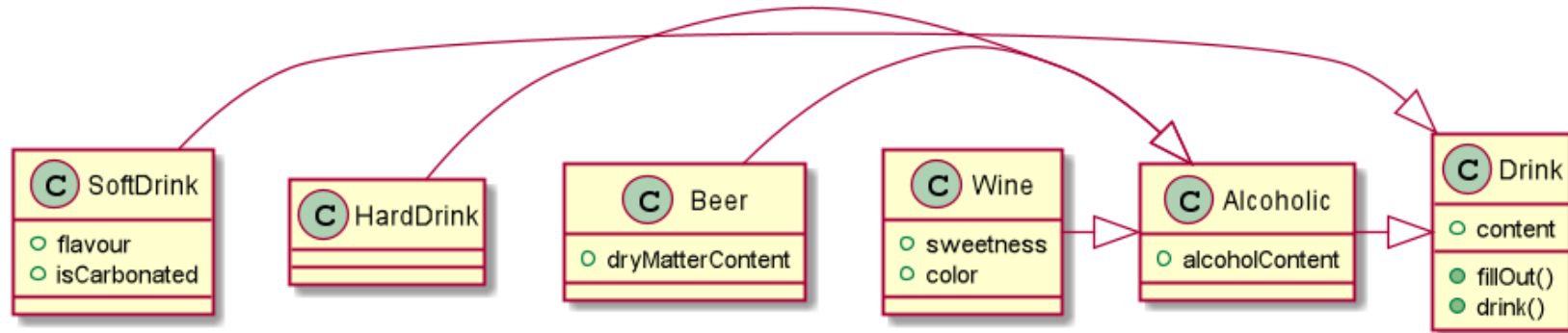
- If a method of the parent class (or the implemented interface) is declared in the child object too then we say that the child object **overrides** that method.
 - In **Java** the overriding methods have to be **invariant** in parameters and **covariant** in exceptions and in the return value.
 - Other languages may enable **contra-variance** in the parameters.

Inheritance

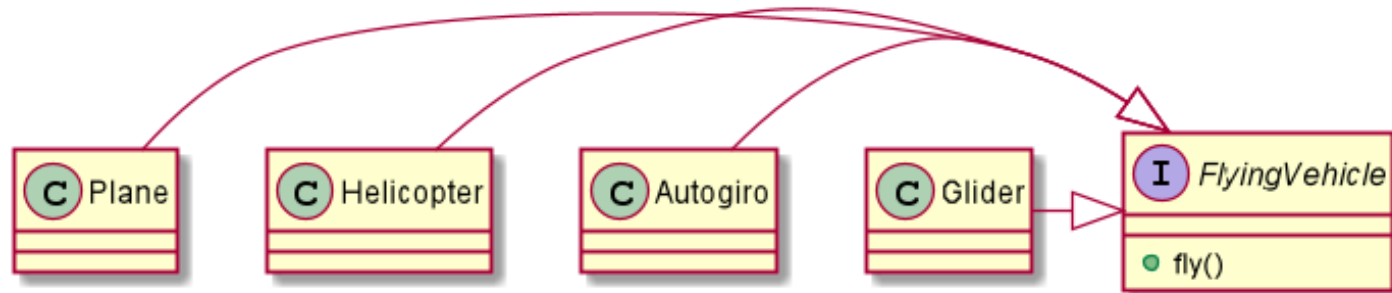
- In **Java** there are two kinds of inheritance.
 - Implementation inheritance (extending classes),
 - Interface inheritance (implementing interfaces).
- The goal of **implementation inheritance** is to inherit the already implemented program code.
- The goal of **interface inheritance** is to describe the contract of an object.

Inheritance

- Implementation inheritance

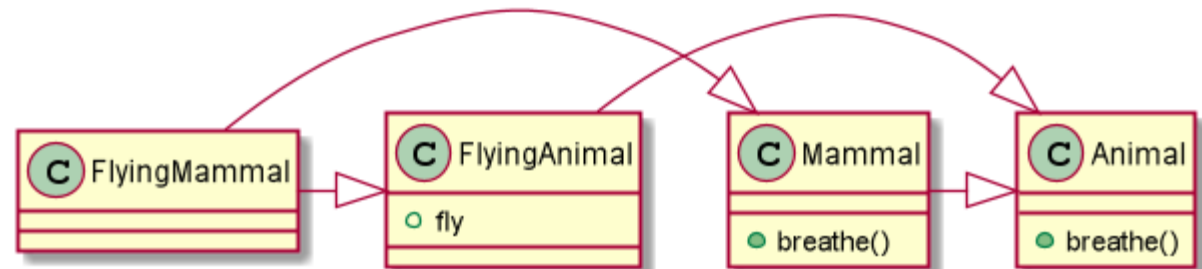


- Interface inheritance



Inheritance

- Languages may support **single or multiple** inheritance.
 - **Multiple inheritance** is supported by C++, and by Java interfaces.
 - Java supports only **single implementation inheritance** and there is an **Object** class, which is the implicit common root class of each object.
 - In the case of implementation inheritance multiple inheritance raises the **diamond problem**.



Inheritance

- The current **rules of thumb** are
 - Interface inheritance should be preferred over implementation inheritance.
 - Composition should be preferred over implementation inheritance.
 - Implementation inheritance should generally avoid.
 - "I'd leave out classes." (James Gossling)

Inheritance

- There may be **other relationships** between objects.
 - **Association** – there is some relationship between the objects.
 - **Composition** – the object **owns** other objects that can't exist alone.
 - **Aggregation** – the object **uses** other objects that can exist without the aggregating object.
- The current **rule of thumb** is that **composition** should be preferred over implementation inheritance.

Polymorphism

- In programming context polymorphism is usually refers as
 - The provision of **a single interface to entities of different types**,
 - The use of **a single symbol to represent multiple different types**.
- The **types** of polymorphism are
 - **Ad-hoc polymorphism** (method overloading),
 - **Parametric** polymorphism (generic programming),
 - **Subtyping and inheritance** (in object-oriented context).

Polymorphism

Ad-hoc Polymorphism

- **Method overloading.**
- That may be
 - **static** (C++, Object Pascal, Java).
 - `substring(startpos[, endpos])`, `indexOf(substring, string[, startpos])`.
 - `System.out.println(int | double | boolean | Object)`.
 - **dynamic** (JavaScript, Groovy).

Polymorphism

Parametric polymorphism

- A set data structure has the following characteristics
 - All elements must be unique (a set cannot store duplicated elements).
 - Operators:
 - Union, intersection, difference.
 - Adding and removing elements.
- The above characteristics are independent from the type of the elements; therefore, we can define a general Set class.

Polymorphism

Parametric polymorphism

- Let's see the following set:
 - { 1, 2, 3.5, "apple", new Person("John Doe", "2000-01-01"), new Car("ABC123") }
 - The elements of different types may lead to runtime errors, for example when the program try to compare the elements or when we attempt to process an element by making assumptions according its type.
- Even if a set is intended to contain homogenous elements
 - Programming mistakes that result in adding elements of other types may happen.
 - The programmer must cast each element of the set before processing that.
- Type declarations are always preferable to type casts.
- Compile time errors are always preferable to runtime exceptions.

Polymorphism

Parametric polymorphism

- Thus, we would like to tell the compiler the exact type of the elements of the set, therefore
 - It could check the types of the elements added to the set, which would prevent runtime exceptions and make type casts unnecessary.
- On the other hand, we wouldn't like to define different classes for different types, since the general characteristics of the sets are common.

Polymorphism

Parametric polymorphism

```
Set<Integer> intSet = new Set<>();  
intSet.add(1);  
intSet.add(0);  
intSet.add("apple");           // compiling error  
for (Integer i : intSet) {     // no type-cast is needed  
    System.out.println(i);  
}
```

Polymorphism

Subtype / inheritance

- If a variable, formal parameter or return value is declared by a type then **a value of any subtypes/inherited type of the declared type can be passed** as value.
- **It is the most important feature of object-orientation**, because of the **replaceability** of the objects.
- If the subclass **overrides** a method of the parent, when that method is called through a polynomial reference then
 - If the method of the parent (the declared type) is called, then the **dispatch is static**.
 - If the method of the subclass (the actual type) is called, then the **dispatch is dynamic**.
 - In C++, virtual only methods are dispatched dynamically, in **Java** static methods are dispatched statically, non-static methods are dispatched dynamically.

Polymorphism

Subtype / inheritance

```
private printAreae(Shape shape) {  
    System.out.println(shape.area());  
}  
  
private static Shape createShape() {  
    Point p1, p2, p3;  
    //  
    Triangle triangle = new Triangle(p1,p2,p3);  
    return triangle;  
}  
  
public static void main(String[] args) {  
    printArea(createShape());  
}
```